PUBLIC

Code Assessment

of the Curve STD Smart Contracts

24 August, 2026

Produced for

Curve

by

CHAINSECURITY

Contents

1 Executive Summary

Dear Curve Team,

Thank you for trusting us to help Curve with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Curve STD according to Scope to support you in forming an opinion on their security risks.

Curve implements a standard library containing reusable modules and interfaces to be used in Vyper smart contract development.

The most critical subjects covered in our audit are the functional correctness of these helpers, interface fidelity to the canonical specs, and consistency of revert/return semantics so that downstream contracts can rely on identical behavior. We found security regarding all these subjects to be high, except for an issue regarding string type bounds in interfaces classified as low severity; see Bounds for interfaces returning String, which was corrected.

Except for the `ema` module, no tests were provided for the library. We strongly recommend building a comprehensive test suite to ensure the correctness of the library code and to prevent regressions in future updates.

In summary, we find that the codebase provides a high level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	1
• Code Corrected	1

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the Curve STD repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

V	Date	Commit Hash	Note
1	17 November 2025	6db238291a62eee43446a0f86323f071e5a001eb	Initial Version
2	14 January 2026	c8025a06d805b4eeb1a50e721d9250841dc1d4ea	Ema
3	18 May 2026	5556df1908837454c1b99d3cb79dd7e3d552d047	Fixes
4	01 July 2026	843d92fbbedc55a71bf7286e460f5a8a3fcae4a8	Fixes
5	12 August 2026	26d4128d5d15cf36a4dccf5ed1599dae651d9042	Fixes

For the Vyper smart contracts, the compiler version `0.4.3` was chosen.

The following files were included in the scope of this review:

```
contracts/interfaces/IERC20.vyi
contracts/interfaces/IERC4626.vyi
contracts/error.vy
contracts/math.vy
contracts/token.vy
```

As of Version 2, the `contracts` directory was renamed to `curve_std`, `error.vy` was removed, and the following files were added to the scope of this review:

```
curve_std/constants.vy
curve_std/ema.vy
```

As of Version 3, `math.vy` was renamed to `crv_math.vy`.

2.1.1 Excluded from scope

Anything not explicitly listed in the scope section above is considered out of scope for this review.

2.2 System Overview

This system overview describes the latest received version (Version 5) of the contracts as defined in the Assessment Overview.

In the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Curve offers a compact Vyper standard library meant to be reused by other smart contracts.

- `IERC20.vyi` and `IERC4626.vyi` define ERC-20 and ERC-4626 interfaces, including ERC-20 metadata functions and the standard ERC-4626 deposit, mint, withdraw and redeem flows.
- `constants.vy` exposes fixed-point constants used by the other modules, including `WAD` and `SWAD` as `10**18`.
- `token.vy` provides reusable ERC-20 helper routines for max approval, `transfer` and `transferFrom`. `max_approve` only sets an unlimited allowance when the current allowance is zero, while the transfer helpers skip zero-amount transfers. All three helpers wrap token calls with `default_return_value=True` to handle tokens that do not return a boolean value.
- `crv_math.vy` exposes `sub_or_zero`, which returns `a - b` when `a >= b` and zero otherwise.
- `ema.vy` implements exponentially weighted moving averages (EMA) with a configurable time period per EMA. Each EMA is identified by a `String[4]` ID and the list of supported IDs is fixed during deployment through `EMAConfig` entries. At most `MAX_EMAS` (10) instances can be configured, duplicate IDs are rejected, and `ema_time` must be non-zero. The allow-list is exposed through the immutable public `ALLOWED_EMAS` getter. The EMA module exposes internal helpers to check whether an ID is allowed, update an EMA, read the current EMA value, and change an EMA time. `read` computes the current value from the elapsed time, the stored `prev_value` and the cached `queued_value` without persisting state. `update` persists the computed EMA value and queues the newly supplied value for the next computation. `set_ema_time` first updates the current EMA using the existing queued value, then stores the new non-zero time.

2.3 Trust Model

This library is meant to be reused by other systems. Therefore, the trust model depends on the usage of the library. Notably, the library has no entry points for end users to interact with.

While the token module provides safe transfer functions, tokens with uncommon behaviors like deflation, inflation, callbacks, fee-on-transfer or rebasing should be handled with care by the integrating contracts.

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Open Findings

In this section, we describe any open findings. Findings that have been resolved have been moved to the Resolved Findings section. The findings are split into these different categories:

- Correctness: Mismatches between specification and implementation

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	0

6 Resolved Findings

Here, we list findings that have been resolved during the course of the engagement. Their categories are explained in the Open Findings section.

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	1

- Bounds for Interfaces Returning String Code Corrected

Informational Findings	4

- Large EMA Values Can Revert Reads Code Corrected
- Missing Events Code Corrected
- Misleading Argument Name Code Corrected
- div_up Overflow Risk Code Corrected

6.1 Bounds for Interfaces Returning String

Correctness Low Version 1 Code Corrected

CS-CURVE-STD-001

In the `IERC20` interface, the following is defined:

```
#NOTE: interface uses `String[1]` where 1 is the lower bound of the string returned by the function.
#    For end-users this means they can't use `implements: ERC20Detailed` unless their implementation
#    uses a value n >= 1. Regardless this is fine as one can't do String[0] where n == 0.

@view
def name() -> String[1]:
    ...


@view
def symbol() -> String[1]:
    ...
```

While this compiles when implementing the interface as `implements: IERC20`, it is not safe to use when the interface is used to type an address and call its functions. For example:

```
token: IERC20

@external
def get_name() -> String[20]:
    return staticcall self.token.name()
```

The above code will fail at runtime if the actual token's name is more than one character long: `compiler: String[1] bounds check`.

Code corrected:

The bounds for the string return types were changed to `NAME_MAX_LEN = 64` and `SYMBOL_MAX_LEN = 32`, respectively.

Note however that contracts implementing the `IERC20` interface using the `implements: IERC20` syntax must adhere to these bounds, meaning that their `name` and `symbol` functions must have a return type of `String[X]` where `X` is at least `NAME_MAX_LEN` and `SYMBOL_MAX_LEN` respectively.

6.2 Large EMA Values Can Revert Reads

Informational **Version 3** **Code Corrected**

CS-CURVE-STD-005

In `ema.vy`, `read` computes the smoothed value with checked `uint256` multiplications:

```
return (ema.prev_value * mul + ema.queued_value * (WAD - mul)) // WAD
```

Since `mul` is scaled by `WAD = 10**18`, very large `prev_value` or `queued_value` values can overflow during this calculation. `update` does not bound `_new_value` before storing it as the next `queued_value`. As a result, if an integration queues a value above the safe range, later `read` or `update` calls with non-zero elapsed time may revert for that EMA.

Code corrected:

Values that enter the EMA in both `update` and the constructor are now ensured to be smaller than or equal to `max_value(uint256) // WAD`.

6.3 Missing Events

Informational **Version 2** **Code Corrected**

CS-CURVE-STD-004

In the `ema` contract, no events are emitted when important actions are performed, such as when an EMA is updated with a new value or when a new `ema` time is set. This makes it difficult for users and developers to track changes and understand the history of EMA updates.

Code corrected:

Events were added to the contract to emit relevant information whenever an EMA is updated, or when an EMA time is set.

6.4 Misleading Argument Name

Informational **Version 1** **Code Corrected**

CS-CURVE-STD-002

In `IERC4626`, `maxDeposit` and `maxMint` have a named argument `owner` which is slightly misleading compared to the ERC-4626 specification, where these functions take a `receiver` argument.

Code corrected:

The argument name was changed to `recipient` in both functions.

6.5 `div_up` Overflow Risk

Informational **Version 1** **Code Corrected**

CS-CURVE-STD-003

The helper `math.div_up` computes `(numerator + denominator - 1) // denominator`. Because the first addition is performed in `uint256` arithmetic, it can overflow when both operands are large (e.g. `numerator = 2**256 - 1`, `denominator = 2`). In such cases, Vyper will revert before the division executes, even though the mathematical result would fit in `uint256`.

Code corrected:

The helper was removed from the codebase.

7 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

7.1 `max_approve` Behavior

Note Version 1

In the `token` module, `max_approve` resets an allowance to the maximum `uint256` value only if the current allowance is 0. Integrators of the library should be aware that this function does not set the allowance unconditionally, which may lead to unexpected behavior if the current allowance is non-zero.